

FACULDADE SENAI FATESG

CURSO SUPERIOR DE TECNOLOGIA EM ANALISE E DESENVOLVIMENTO DE
SISTEMAS

STEFANE PIRES LIMA, DANIEL ALVES CARVALHO E FRANCISCO AIRTON N. FILHO,
RYAN DA SILVA DORIGUETTO

PLATAFORMA DE TREINOS MOBILE: GYMCONNECT

Goiania, 2026

SUMARIO

1. INTRODUCAO.....	3
2. ESCOPO DO PROJETO.....	3
3. TERMO DE ABERTURA DO PROJETO - TAP.....	3
4. ESTRUTURA ANALITICA DO PROJETO - EAP.....	6
5. ESPECIFICACAO DE REQUISITOS.....	7
6. DIAGRAMAS.....	8
7. AMBIENTE DE PRODUCAO.....	8
8. ESPECIFICACAO DE INTERFACE.....	9
9. DESCRICAO DO DESENVOLVIMENTO DO SISTEMA PROPOSTO.....	10
10. CONSIDERACOES FINAIS.....	15
REFERENCIAS.....	15
Ficha Tecnica do Sistema - GymConnect (Versao Resumida - ABNT).....	17

1. INTRODUCAO

Este documento apresenta o Projeto Final da plataforma digital de treinos para academias, cujo objetivo e permitir que administradores cadastrem e organizem treinos semanais, com exercicios detalhados e videos explicativos, enquanto o aluno tem acesso facil ao seu cronograma diario e semanal.

O sistema surge da necessidade das academias em otimizar o acompanhamento remoto dos alunos, garantir seguranca na execucao de exercicios e facilitar a organizacao individualizada dos treinos.

A plataforma oferece:

- Treinos personalizados
- Catalogo de exercicios com descricao e video
- Videos integrados via YouTube
- Assistente virtual de IA (GIA) para duvidas sobre treinos e nutricao
- Acesso offline ao cronograma e treinos (cache local)
- Fluxo intuitivo para administrador (Professor) e aluno

2. ESCOPO DO PROJETO

O projeto abrancara o desenvolvimento das seguintes funcionalidades:

- Sistema de login e cadastro para alunos e administradores.
- Painel administrativo mobile para cadastro de alunos e criacao de cronogramas semanais.
- Area do aluno com acesso ao seu treino do dia e calendario semanal.
- Videos explicativos vinculados a cada exercicio.
- Catalogo de exercicios com possibilidade de adicionar e editar novos.
- Assistente virtual de IA (GIA), integrada ao Google Gemini API.
- Acesso offline ao cronograma e treinos via cache local (SQLite).

3. TERMO DE ABERTURA DO PROJETO - TAP

3.1. Justificativa

As academias enfrentam dificuldades em organizar treinos personalizados e acompanhar a execucao correta dos exercicios sem supervisao continua. Com o crescimento das solucoes digitais moveis, torna-se essencial oferecer uma plataforma mobile-first, acessivel em

smartphones Android e iOS, que centralize treinos, facilite o acompanhamento e forneça vídeos explicativos confiáveis.

3.2. Objetivos do Projeto

Objetivo Geral: Desenvolver um aplicativo mobile multiplataforma para academias administrarem treinos personalizados, disponibilizando aos alunos cronogramas semanais com vídeos explicativos e suporte de assistente virtual de IA.

Objetivos Específicos:

- Criar sistema de login e autenticação via aplicativo.
- Permitir cadastro de alunos e criação de cronogramas semanais.
- Exibir ao aluno seu treino diário e semanal em interface responsiva.
- Integrar vídeos explicativos por exercício via YouTube API.
- Disponibilizar assistente virtual de IA (GIA) para consultas sobre treinos e exercícios.
- Garantir acesso offline ao cronograma e treinos via cache local.
- Garantir desempenho mínimo de carregamento (< 5 segundos).

3.3 Escopo do Projeto

Incluso

- Cadastro/login de usuários (aluno e administrador).
- Painel administrativo mobile para gerenciar alunos e treinos.
- Cronograma semanal e treino do dia acessíveis pelo app.
- Catálogo de exercícios com possibilidade de adicionar novos.
- Player de vídeos explicativos integrado (YouTube).
- Assistente virtual de IA (GIA) integrada ao Google Gemini API.
- Acesso offline ao cronograma e treinos (cache local via SQLite).

Não Incluso (futuro)

- Dieta personalizada.
- Integração com wearables.
- Notificações push.

3.4 Premissas

- O aluno precisa ter internet para assistir aos vídeos e para interagir com a GIA.
- O cronograma e os treinos ficam disponíveis offline via cache local, com sincronização automática ao reconectar.

- A academia e responsavel por cadastrar e manter seu proprio conteudo atualizado.
- Os videos sao fornecidos pelo administrador e hospedados externamente (YouTube).
- A variavel de ambiente GEMINI_API_KEY deve estar configurada no servidor backend para o funcionamento da GIA.

3.5. Restricoes

- Tempo de resposta inferior a 5 segundos para carregamento dos treinos.
- Uso obrigatorio de links externos para videos (nao armazenados localmente).
- O MVP sera entregue ate o final do semestre conforme planejamento docente.

3.6. Riscos do Projeto

Risco	Probabilidade	Impacto	Estrategia
Falha na integracao de videos	Media	Media	Usar embed padrao do YouTube
Lentidao no carregamento	Baixa	Alta	Otimizar consultas e cache local SQLite
Mudanca de requisitos	Media	Alta	Reunioes semanais com orientador
Conteudos inseridos incorretamente pela academia	Alta	Baixa	Validacao e campos obrigatorios
Indisponibilidade da API Gemini GIA	Baixa	Media	Mensagem amigavel ao usuario

3.7. Stakeholders

- **Aluno** - consumidor dos treinos.
- **Academia/Administrador** - responsavel por criar treinos.
- **Equipe de desenvolvimento** - responsavel pela implementacao.
- **Professor orientador** - acompanhamento academico.

3.8. Cronograma Resumido

- Fevereiro - Levantamento inicial e TAP/EAP.
- Marco - Definicao de requisitos e modelagem.
- Abril - Desenvolvimento backend e integracao inicial Flutter.
- Maio - Desenvolvimento frontend mobile e testes.
- Junho - Validacao final e entrega do MVP mobile.

4. ESTRUTURA ANALITICA DO PROJETO - EAP

Nível 1 - Projeto Aplicativo Mobile GymConnect

4.1. Gerenciamento do Projeto

- 4.1.1 Definicao do escopo
- 4.1.2 Atualizacao do TAP
- 4.1.3 Atualizacao da EAP
- 4.1.4 Planejamento do cronograma
- 4.1.5 Acompanhamento com o orientador

4.2. Analise e Levantamento de Requisitos

- 4.2.1 Requisitos Funcionais
- 4.2.2 Requisitos Nao Funcionais
- 4.2.3 Personas e historias de usuario
- 4.2.4 Fluxos do sistema (login, treino, player de video, GIA)

4.3. Modelagem do Sistema

- 4.3.1 Modelagem do Banco (MER/DER)
- 4.3.2 Diagrama de Casos de Uso
- 4.3.3 Diagrama de Classes
- 4.3.4 Diagrama de Navegacao (fluxo aluno e professor)

4.4 Backend

- 4.4.1 Criacao do banco de dados
- 4.4.2 API de autenticacao (JWT + Spring Security)
- 4.4.3 API de cadastros (aluno, exercicios, treinos)
- 4.4.4 API de cronograma semanal
- 4.4.5 Integracao dos videos via link
- 4.4.6 Integracao com Google Gemini API (assistente GIA)
- 4.4.7 API de historico de chat por usuario

4.5 Frontend - Professor (Administrador)

- 4.5.1 Tela de login
- 4.5.2 Tela inicial (catalogo de exercicios)
- 4.5.3 Cadastro de alunos
- 4.5.4 Cadastro e edicao de treinos

- 4.5.5 Consulta ao catalogo de exercicios

4.6 Frontend - Aluno

- 4.6.1 Dashboard (treino do dia e atividades recentes)
- 4.6.2 Cronograma semanal com marcacao de exercicios
- 4.6.3 Player de videos e descricao de execucao
- 4.6.4 Navegacao entre dias da semana
- 4.6.5 Chat com assistente GIA
- 4.6.6 Tela de perfil

4.7 Testes

- 4.7.1 Testes unitarios (Flutter + backend)
- 4.7.2 Testes de usabilidade
- 4.7.3 Testes de desempenho
- 4.7.4 Ajustes pos-testes

4.8. Entregaveis

- 4.8.1 MVP mobile funcional (APK/IPA)
- 4.8.2 Documentacao final
- 4.8.3 Manual do Usuario

4.9 Encerramento

- 4.9.1 Validacao final com o professor
- 4.9.2 Apresentacao
- 4.9.3 Entrega dos documentos oficiais

5. ESPECIFICACAO DE REQUISITOS

5.1 Requisitos Funcionais

RF01 - O Aluno deve realizar o proprio cadastro.

RF02 - O administrador deve criar treinos semanais.

RF03 - O aluno deve realizar login com e-mail e senha.

RF04 - O aluno deve visualizar o treino do dia.

RF05 - O aluno deve visualizar o cronograma semanal.

RF06 - O aluno deve assistir aos videos explicativos.

RF07 - O aluno deve visualizar seu perfil com dados basicos e historico de treinos.

RF08 - O aluno deve poder interagir com a assistente virtual GIA para tirar duvidas sobre treinos, exercicios e orientacoes de saude.

RF09 - O aplicativo deve disponibilizar o cronograma e os treinos no modo offline, sincronizando automaticamente ao reconectar a internet.

5.2 Requisitos Nao Funcionais

RNF01 - Carregar cronograma < 5 segundos.

RNF02 - A interface deve ser adaptada para diferentes tamanhos de tela (smartphones e tablets).

RNF03 - Interfaces devem ser intuitivas, permitindo uso sem treinamento previo.

RNF04 - Codigo documentado.

RNF05 - Facilidade de uso e aprendizado.

RNF06 - O sistema deve funcionar em modo offline para consulta de treinos, exibindo aviso claro quando sem conexao.

6. DIAGRAMAS

6.1 Diagrama de Classe

[Figura 1 - Diagrama de Classe GymConnect]

A Figura 1 apresenta o diagrama de classes do GymConnect, evidenciando as principais entidades do sistema e seus relacionamentos: usuarios, perfil, cronograma, exercicios, cronograma_exercicio e cronograma_execucao.

Fonte: Elaborado pela equipe (2026)

6.2 Diagrama de Arquitetura e Solucoes

[Figura 2 - Diagrama de Arquitetura e Solucoes - VERSAO CORRIGIDA]

A Figura 2 apresenta a arquitetura geral da solucao. A arquitetura implementada e composta pelos seguintes elementos:

- Camada Cliente: Aplicativo Mobile Flutter (Android/iOS). Nao existe painel web administrativo; o Professor utiliza o mesmo aplicativo mobile com telas exclusivas de gestao.
- Camada de Servicos / Backend: API REST em Spring Boot com Spring Security (JWT) e

integracao ao Google Gemini API para o assistente GIA.

- Servicos Externos: YouTube API (videos explicativos) e Google Gemini API (IA generativa para a assistente GIA).
- Camada de Dados: MySQL 8.x no servidor (via Docker) e SQLite local no dispositivo mobile para cache offline de treinos e sessao do usuario.
- Banco de Dados: tabelas usuarios, perfil, cronograma, exercicios, cronograma_exercicio, cronograma_execucao.

CORRECOES EM RELACAO AO DIAGRAMA ORIGINAL: O componente 'Supabase' deve ser removido (nao utilizado; o banco e MySQL). O componente 'Painel Web Administrativo' deve ser removido (nao existe; o Professor usa o app mobile). O 'Storage de Imagens' ainda nao esta implementado; fotos de perfil sao salvas localmente no dispositivo.

Fonte: Elaborado pela equipe (2026)

7. AMBIENTE DE PRODUCAO

Cliente (usuario final):

- Dispositivos compatíveis: Smartphones e tablets Android (versao ≥ 8.0) e iOS (versao ≥ 13).
- Requisitos mínimos: processador dual-core, memoria RAM de 2 GB.
- Conexao com internet (necessaria para videos e GIA; cronograma e treinos funcionam offline).
- Armazenamento disponivel de 200 MB para instalacao do aplicativo.
- Aplicativo: Desenvolvido em Flutter, distribuido em formato APK (Android) e IPA (iOS).
- Funcionalidades locais com cache offline: login, visualizacao de treinos, cronograma semanal, player de video integrado.

Servidor Backend:

- Processador: 2 nucleos.
- Memoria RAM: 2 GB.
- Java 21 instalado.
- Docker instalado (para execucao do MySQL via container).
- Variavel de ambiente GEMINI_API_KEY configurada com chave valida do Google Gemini API.

8. ESPECIFICACAO DE INTERFACE

8.1 Tela de Login

- Funcao: Permitir acesso de alunos e professores (administradores).
- Elementos: Campos de e-mail e senha, botao 'Entrar', link 'Criar Conta'.
- Comportamento: Apos autenticao via JWT, o sistema redireciona para o painel correspondente ao tipo de perfil.

8.2 Tela de Cadastro de Conta

- Funcao: Criar novos perfis de usuario (Aluno ou Professor).
- Elementos: Seletor de tipo de conta (Aluno / Professor), campos obrigatorios (Nome completo, E-mail, Senha, Confirmacao de senha), checkbox de aceite dos termos de uso, botao 'Cadastrar'.
- Comportamento: Validacao dos dados e criacao do perfil no banco de dados.

8.3 Painel do Professor (Administrador)

A interface do GymConnect Mobile foi projetada para oferecer uma experiencia intuitiva e responsiva em dispositivos Android e iOS, priorizando usabilidade, acessibilidade e clareza na navegacao.

- Funcao: Gerenciar alunos, exercicios e treinos.
- Elementos: Menu de navegacao inferior com abas Exercicios, Treinos, Alunos e Perfil.
- Comportamento: Permite cadastrar, editar e excluir exercicios; criar e gerenciar treinos; visualizar e remover alunos.

8.4 Tela de Gerenciamento de Treinos

- Funcao: Criar e editar treinos personalizados.
- Elementos: Aba 'Treinos Cadastrados', botao 'Criar Treino', campos de configuracao (aluno, dia da semana, exercicio, series, repeticoes, carga).
- Comportamento: O administrador define exercicios e salva o treino, vinculado ao cronograma do aluno.

8.5 Tela do Aluno - Cronograma Semanal

- Funcao: Exibir treinos programados para a semana.
- Elementos: Saudacao com nome do aluno, cartoes com dias da semana, quantidade de exercicios e botao 'Treinar'.
- Comportamento: Ao clicar em 'Treinar', o aluno acessa o treino do dia e videos explicativos.

8.6 Tela de Perfil do Aluno

- Funcao: Mostrar informacoes pessoais e historico de treinos.
- Elementos: Nome, e-mail, data de cadastro, lista de treinos concluidos, botao 'Editar Perfil'.
- Comportamento: O aluno pode atualizar seus dados e acompanhar seu progresso.

9. DESCRICAO DO DESENVOLVIMENTO DO SISTEMA PROPOSTO

O desenvolvimento do GymConnect Mobile foi orientado pela necessidade de integrar funcionalidades administrativas e pedagogicas em um unico aplicativo multiplataforma, garantindo escalabilidade, seguranca e usabilidade. A aplicacao foi construida em Flutter, consumindo servicos disponibilizados por uma API REST desenvolvida em Spring Boot, com persistencia de dados em MySQL. O backend implementa autenticao via JWT (Spring Security + java-jwt) e integra o Google Gemini API para o assistente virtual GIA. O mobile adota arquitetura Offline-First com cache local em SQLite, garantindo acesso ao cronograma e treinos mesmo sem conexao com a internet.

9.1. Telas de Login e de Cadastro de Conta (Usuario: Aluno / Academia)

[Figura - Capturas de tela das telas de Login e Criar Conta]

Funcao

Permitir que usuarios realizem cadastro e login no sistema GymConnect para acessar as funcionalidades do aplicativo.

Elementos

Campos de nome completo, e-mail, senha e confirmacao de senha, selecao de tipo de perfil (Aluno ou Professor), checkbox de aceite dos termos de uso, botao 'Criar Conta', campos de login (e-mail e senha), botao 'Entrar', opcao para visualizar/ocultar senha e links de navegacao entre cadastro e login.

Comportamento

No cadastro, o usuario preenche os dados solicitados, seleciona o tipo de perfil e aceita os termos de uso. Ao clicar em 'Criar Conta', as informacoes sao validadas e armazenadas no banco de dados. Na tela de login, o usuario informa e-mail e senha para autenticao via JWT. Apos a validacao das credenciais, o sistema libera o acesso a plataforma e direciona o usuario para a tela inicial correspondente ao seu perfil.

9.2. Tela Inicial (Professor)

[Figura - Captura de tela da Tela Inicial do Professor]

Funcao

Permitir que o administrador visualize e gerencie o catalogo de exercicios cadastrados.

Elementos

Lista de exercicios com nome, indicador de video e descricao, botao de opcoes (editar/excluir) para cada exercicio, botao '+Novo' para adicionar exercicio e menu de navegacao inferior com abas Exercicios, Treinos, Alunos e Perfil.

Comportamento

Ao acessar a tela, o sistema exibe o catalogo de exercicios. O administrador pode visualizar os detalhes de cada exercicio, acessar opcoes de gerenciamento e criar novos exercicios atraves do botao '+Novo'.

9.3. Telas de Cadastro de Novo Treino

[Figura - Capturas de tela das telas de Treinos e Criar Treino]

Funcao

Permitir que o administrador crie e personalize treinos para os alunos, definindo exercicios, dias da semana, series, repeticoes e carga.

Elementos

Campo de selecao de aluno, selecao do dia da semana, selecao de exercicios do catalogo, campos de series, repeticoes e carga, botao 'Adicionar exercicio', botao 'Adicionar dia' e botao 'Salvar Treinos'.

Comportamento

O administrador seleciona um aluno e monta o treino adicionando exercicios e suas configuracoes. E possivel cadastrar multiplos exercicios para diferentes dias da semana. Ao clicar em 'Salvar Treinos', as informacoes sao armazenadas no sistema e vinculadas ao aluno selecionado.

9.4. Tela de Gerenciamento de Alunos

[Figura - Captura de tela da Tela de Gerenciamento de Alunos]

Funcao

Permitir ao administrador visualizar e gerenciar os alunos cadastrados no sistema.

Elementos

Lista de alunos cadastrados, foto ou inicial do usuario, nome, e-mail, botao de exclusao e menu de navegacao inferior.

Comportamento

Ao acessar a tela, o sistema exibe todos os alunos cadastrados. O administrador pode visualizar as informacoes de cada aluno e remover um cadastro ao selecionar o icone de exclusao. A lista e atualizada automaticamente apos a remocao.

9.5. Tela do Aluno Inicial

[Figura - Captura de tela do Dashboard do Aluno]

Funcao

Apresentar a tela inicial do aplicativo, permitindo acesso rapido aos treinos, perfil do usuario e assistente virtual GIA, alem de exibir o historico de atividades recentes.

Elementos

Mensagem de boas-vindas, card de acesso aos treinos, card de perfil, card da assistente virtual GIA, secao de atividades recentes (atualizada conforme exercicios e treinos sao concluidos), botoes de configuracoes e logout, alem do menu de navegacao inferior.

Comportamento

Ao selecionar uma das opcoes da dashboard, o usuario e direcionado para a funcionalidade correspondente. A secao de atividades recentes e atualizada automaticamente conforme os exercicios e treinos forem concluidos. O historico e persistido localmente por usuario via SQLite.

9.6 Tela do Aluno - Cronograma Semanal

[Figura - Captura de tela do Cronograma Semanal do Aluno]

Funcao

Exibir os exercicios do treino diario do aluno, permitindo acompanhar a execucao, visualizar detalhes e marcar o exercicio como concluido.

Elementos

Lista de exercicios, nome do treino, quantidade de exercicios, video demonstrativo via YouTube Player, secao 'Como executar' com descricao detalhada, informacoes de series,

repeticoes e carga, botao 'Marcar como Feito' e menu de navegacao inferior.

Comportamento

Ao selecionar um exercicio, o usuario pode visualizar a demonstracao em video e instrucoes de execucao. Ao clicar em 'Marcar como Feito', o exercicio e registrado como concluido no treino do aluno. As conclusoes sao persistidas localmente e sincronizadas com o backend.

9.7 Tela do ChatBot (GIA)

[Figura - Captura de tela da GIA]

Funcao

Permitir que o aluno interaja com a assistente virtual GIA para consultar treinos, tirar duvidas sobre exercicios e receber orientacoes personalizadas.

Elementos

Campo de mensagem, botao de envio, area de conversa em formato de chat com historico persistido por usuario, icone da GIA nas respostas da assistente e menu de navegacao inferior.

Comportamento

Ao enviar uma mensagem, o backend encaminha a solicitacao ao Google Gemini API. A assistente virtual processa a solicitacao e retorna informacoes relacionadas aos treinos, exercicios e orientacoes do usuario. O historico de chat e persistido por usuario tanto no backend (endpoint dedicado) quanto localmente. Em caso de indisponibilidade da API por sobrecarga, o sistema exibe mensagem amigavel ao usuario.

10. CONSIDERACOES FINAIS

O desenvolvimento do GymConnect Mobile demonstrou a viabilidade de criar uma solucao multiplataforma voltada para academias, integrando funcionalidades administrativas e pedagogicas em um unico aplicativo. A utilizacao do Flutter para o frontend, em conjunto com Spring Boot e MySQL no backend, garantiu escalabilidade, responsividade e seguranca, atendendo aos requisitos funcionais e nao funcionais definidos. A integracao com o Google Gemini API para o assistente virtual GIA e a implementacao da arquitetura Offline-First com SQLite representam diferenciais tecnologicos relevantes do projeto.

Os resultados preliminares indicaram que o sistema oferece maior organizacao e praticidade no gerenciamento de treinos, reduzindo o tempo de administracao e ampliando a autonomia dos alunos. Alem disso, a integracao de videos explicativos contribuiu para a execucao

correta dos exercicios, promovendo maior engajamento e adesao as rotinas de treino. O assistente GIA agrega valor ao permitir que os alunos tirem duvidas sobre treinos e nutricao diretamente pelo aplicativo.

Apesar dos avancos, algumas limitacoes foram identificadas, como a ausencia de notificacoes push (prevista para implementacao futura), a necessidade de integracao com sistemas nutricionais e dispositivos vestiveis, e o armazenamento de fotos de perfil ainda local (nao em servico de nuvem).

Como trabalhos futuros, propoe-se:

- Integracao com wearables para coleta automatica de metricas fisicas.
- Notificacoes push inteligentes para lembretes personalizados de treino.
- Modulo nutricional integrado ao perfil do aluno.
- Armazenamento de fotos de perfil em servico de armazenamento em nuvem.
- Expansao das capacidades da GIA com contexto personalizado por aluno.

Conclui-se que o GymConnect Mobile contribui para a modernizacao das academias brasileiras, posicionando-se como uma solucao inovadora que une gestao administrativa e acompanhamento pedagogico em um unico sistema. O projeto reforca a importancia da transformacao digital no setor fitness e abre espaco para pesquisas futuras sobre personalizacao e integracao tecnologica.

REFERENCIAS

AMMETTI, F. Flutter na pratica. Traducao de Aldir Coelho Correa da Silva. Sao Paulo: Novatec, 2020.

CAPES. Portal de Periodicos: Usabilidade e acessibilidade em plataformas digitais. Brasilia: CAPES, 2024. Disponivel em: <https://www.periodicos.capes.gov.br/>. Acesso em: 01 jun. 2026.

CARRERA DOS SANTOS, R. Impactos do uso de aplicativos fitness por profissionais de educacao fisica. Universidade Federal do Para, 2020.

CASTRO, Marco Aurelio A.; DUFLOTH, Simone C. Usabilidade no acesso mobile a servicos e informacoes de portais governamentais. Revista Gestao & Tecnologia, v.23, n.2, p.129-159, 2023.

FIELDING, Roy Thomas. Architectural Styles and the Design of Network-Based Software Architectures. 2000.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

GABRIEL, N. M. B. Flutter - Vale a pena investir nesse framework? Iteris Blog, 2022. Disponível em: <https://blog.iteris.com.br/flutter-vale-a-pena-investir-nesse-framework/>. Acesso em: 01 jun. 2026.

GOOGLE. Flutter architectural overview. Disponível em: <https://docs.flutter.dev/resources/architectural-overview>. Acesso em: 01 jun. 2026.

GOOGLE. Gemini API documentation. Disponível em: <https://ai.google.dev/gemini-api/docs>. Acesso em: 01 jun. 2026.

NIELSEN, Jakob. Usability Engineering. San Francisco: Morgan Kaufmann, 1993.

PANOSSO, A.; CERUTTI, F. Aplicativos de atividade física: análise crítica e lacunas científicas. Universidade Federal de Santa Maria, 2021.

RISSI, Matheus; DALLILO, Felipe Diniz. Flutter: um framework para desenvolvimento mobile. RECIMA21, v.3, n.11, p.1-12, 2022. DOI: <https://doi.org/10.47820/recima21.v3i11.2230>.

SALVADOR, Lilian das Mercês; ARAUJO, Tiago Maritan Ugulino de. Técnicas para avaliação de usabilidade em aplicações de dispositivos móveis. Revista GEMInIS, v.14, n.1, p.71-84, 2023.

SILVA, J.; COSTA, M. Aplicativos fitness e mudança de comportamento: uma análise no contexto brasileiro. Universidade de Pernambuco, 2022.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. Pearson, 2019.

YOUTUBE DATA API. Official documentation. Disponível em: <https://developers.google.com/youtube/v3>. Acesso em: 13 maio 2026.

Ficha Tecnica do Sistema - GymConnect (Versao Resumida - ABNT)

1. Informacoes Gerais

- Arquitetura: Cliente-Servidor, baseada em API REST, com divisao em camadas (Controller, Service, Repository, Domain).
- Padroes Utilizados: RESTful, DTOs, RBAC (controle de acesso baseado em papeis), JWT, Offline-First (cache local SQLite).
- Plataforma: Aplicativo mobile multiplataforma (Android/iOS) desenvolvido em Flutter.

2. Tecnologias Utilizadas

2.1 Backend

- Framework: Spring Boot 3.5.6
- Linguagem: Java 21
- Gerenciamento de Dependencias: Maven
- Banco de Dados: MySQL 8.x (via Docker)
- ORM: JPA/Hibernate

Dependencias Principais:

- Spring Boot Starter Web
- Spring Boot Starter Data JPA
- Spring Boot Starter Security
- Spring Boot Starter Test
- Spring Boot DevTools
- MySQL Connector/J
- Lombok
- java-jwt (com.auth0) - autenticacao JWT
- springdoc-openapi-starter-webmvc-ui - documentacao Swagger (/swagger-ui.html)

Integracao Externa:

- Google Gemini API (generativelanguage.googleapis.com) - assistente virtual GIA
- Variavel de ambiente obrigatoria: GEMINI_API_KEY

Plugins:

- Maven Compiler Plugin
- Spring Boot Maven Plugin

2.2 Frontend (Mobile)

- Framework: Flutter 3.x
- Linguagem: Dart
- Gerenciamento de Dependencias: Pub

Bibliotecas Principais:

- provider - gerenciamento de estado (ChangeNotifier)
- dio - requisicoes HTTP com interceptors JWT
- go_router - navegacao declarativa com rotas protegidas
- shared_preferences - persistencia local de sessao/JWT
- youtube_player_flutter - player de video embutido (YouTube)
- intl - formatacao de datas
- image_picker - selecao de foto de perfil (camera/galeria)
- sqflite + path - banco SQLite local (Offline-First)
- connectivity_plus - deteccao de conectividade de rede

3. Ambiente de Desenvolvimento

- Backend: Java 21, Maven 3.9 ou superior, Docker (MySQL via container).
- Frontend: Flutter SDK 3.x, Dart, Android Studio e Xcode (para build iOS).
- IDEs recomendadas: VS Code, IntelliJ IDEA, Android Studio.
- Controle de versao: Git e GitHub.

4. Execucao do Sistema

- Banco de dados: docker-compose up -d (container MySQL)
- Backend: mvn spring-boot:run (requer GEMINI_API_KEY configurada no ambiente)
- Frontend: flutter pub get e depois flutter run

5. Requisitos de Sistema

Servidor Backend

- Processador: 2 nucleos
- Memoria RAM: 2 GB
- Java 21 instalado
- Docker instalado (MySQL via container)
- Chave de API Google Gemini configurada (GEMINI_API_KEY)

Cliente Mobile

- Dispositivo Android ($\geq$ versao 8.0) ou iOS ($\geq$ versao 13)
- Memoria RAM: 2 GB
- Conexao com internet (necessaria para videos e GIA; cronograma funciona offline)

6. Padroes de Desenvolvimento

- Arquitetura em camadas (Controller, Service, Repository)
- Utilizacao de DTOs para transferencia de dados
- Organizacao modular do frontend em Flutter (por modulo: auth, treinos, chat, perfil, cliente, home)
- Arquitetura Offline-First no mobile: SQLite para cache de treinos e sessao local
- Autenticacao via JWT (Spring Security + java-jwt)
- Versionamento via Git e GitHub
- API estruturada conforme boas praticas REST
- Documentacao automatica da API via Swagger (springdoc-openapi) em /swagger-ui.html
- Testes automatizados unitarios e validacao em dispositivos fisicos e emuladores